SupplyForge — capacity factors & data-source interactions

How the load factors are built from each dataset, and how the packages fit together

Simon Brigode – simon.brigode@ehess.fr

11 June 2026

Abstract

A capacity factor (*facteur de charge*, generation side) is the hourly ratio, in $[0, 1]$, of a technology's output to its nameplate capacity — the availability signal that drives wind, solar and run-of-river in the optimisation. SupplyForge can build it from four data sources, each with a different method but a single shared output schema. This note explains that construction source by source, the reservoir-inflow reconstruction, and the source-selection machinery; then the demand-side mirror — how DemandForge constructs the load curve and where the *load factor* (average over peak load) comes from; and finally how the SupplyForge (supply) and DemandForge (demand) packages fetch data and interact.

Contents

1 One output, four sources

Every source emits the *same* canonical file — `results/capacity_factors/capacity_factors_{country}_{ye` — with columns `hour`, `plant_type`, `WS` (weather/climate-year tag), `capacity_factor`. Because the schema is fixed, the model builder (`create_pommes_craft_model` / `grid_model`) never changes when the source does; it just reads the file and injects `capacity_factor` as a technology's hourly `availability`. The four sources (Table 1) differ only in *how* the number is produced.

Table 1: Capacity-factor construction by source.

Source	What it is	How CF is built	Status / note
ENTSO-E	observed generation + installed capacity (a real historical year)	*measured*: CF = generation/installed capacity, hourly	default; may exceed 1 on data noise (not clipped)
ERAA 2024	the ERAA prospective dashboard CFs	already a CF: extract + reshape; 35 weather scenarios (WS1–WS35), horizon years	prospective; one file per horizon year
PECD 4.2	CDS climate product; CF at bidding-zone level	*spatial aggregation*: capacity-weighted zone→country (Eq. 3)	the default for the regional/climate models; clipped to $[0, 1]$
ERA5 (raw)	raw ERA5 climate fields	`atlite` conversion + Global-Wind-Atlas bias correction	Phase 2 — not yet wired (`NotImplementedError`)

2 Construction, source by source

2.1 ENTSO-E — measured (the default)

For each intermittent plant type p (Solar, Wind Onshore, Wind Offshore, Hydro Run-of-river) and hour t:

$$\mathrm{CF}_{p,t} = \frac{G_{p,t}}{C_p^{\mathrm{inst}}} \quad (\text{else } 0 \text{ if } C_p^{\mathrm{inst}} = 0), \tag{1}$$

with G the ENTSO-E actual generation and C^{inst} the nationally-aggregated installed capacity. Inputs: `generation_*` + `installed_capacities_*`; the series is resampled to hourly UTC. *Dispatchable* plants instead get an **availability** series from outage events,

$$a_{p,t} = 1 - \frac{U_{p,t}}{C_p^{\mathrm{inst}}}, \qquad U_e = \text{nominal power}_e - \text{available}_e, \tag{2}$$

($U_{p,t}$ = sum of overlapping outages), written to `results/availability/`. This path reflects what the *actual* fleet did in a real weather + outage year.

2.2 ERAA 2024 — prospective scenarios

The ERAA dashboard already publishes capacity factors, so SupplyForge only *extracts and reshapes* them: it reads the staged ERAA CSVs (skipping the metadata header), unpivots the 35 weather-scenario columns into a long `(hour, WS, capacity_factor)` frame, labels the technology, and averages if a technology has multiple files. The `WS` column (`WS1`...`WS35`) is the climate-year axis; horizon years are 2026/28/30/35. No division is needed — it is already a CF.

2.3 PECD 4.2 — capacity-weighted zonal aggregation

PECD ships CF directly for solar (`SPV`), onshore wind (`WON`) and offshore wind (`WOF`), but at the *bidding-zone* level. The construction is therefore **spatial aggregation** to the country: the national CF is the installed-capacity-weighted average over the country's zones z,

$$\mathrm{CF}_{\mathrm{nat}}(t) = \frac{\sum_z \mathrm{CF}_z(t)\, C_z}{\sum_z C_z}. \tag{3}$$

Weights C_z come from a configured per-zone capacity table (config dict → CSV → equal-weight fallback with a loud warning). **Run-of-river** is the exception: PECD gives *weekly* generation energy (`HRO`+`HPO`), so the CF is reconstructed as

$$\mathrm{CF}_{\mathrm{RoR}}(w) = \frac{\mathrm{HRO}_w + \mathrm{HPO}_w}{C_{\mathrm{RoR}}^{\mathrm{inst}} \times 168\,\mathrm{h}}, \tag{4}$$

then interpolated weekly→hourly (the sub-weekly variability is lost — flagged). All series are leap-trimmed (drop Feb 29 ⇒ exactly 8760 h) and clipped to $[0, 1]$.

2.4 ERA5 (raw) — atlite, Phase 2

The raw-ERA5 engine would build an `atlite` cutout from ERA5 climate fields, apply turbine power-curves / a PV model, aggregate grid cells to the country with land-use masks, and correct the wind bias against the Global Wind Atlas. It emits the same canonical schema, but is not yet production-wired (the dispatcher raises `NotImplementedError`).

3 Reservoir hydro inflow — two methods

The reservoir is modelled as a storage fed by a must-run *inflow* signal. The inflow is built two ways and written to `results/inflow/` (`inflow_MW`, `timestamp`):

- **ENTSO-E water balance** — weekly inflow = weekly generation + weekly storage change, $I_w = G_w + \Delta S_w$, interpolated weekly→hourly, clipped ≥ 0, divided by 168 to get MW. It mixes natural inflow with dispatch decisions.
- **PECD `HRI`** — a *natural*-inflow reconstruction (a Random-Forest model of weekly temperature + precipitation), summed across the country's zones, interpolated weekly→hourly. More physical, since it is not contaminated by dispatch.

Downstream the inflow is *shape-normalised* (availability = inflow/ max inflow), so only the temporal *profile* matters — the absolute scale cancels.

4 Source selection & orchestration

Two **orthogonal** config switches (in `sources.py`) choose the source, independently for renewables and for hydro:

```
res_source:   entsoe | era5 | pecd      # wind & solar capacity factors
hydro_source: auto   | entsoe | pecd    # reservoir inflow + run-of-river
#   auto -> follows res_source where it can supply hydro (pecd->pecd), else entsoe
#           (in particular era5 -> entsoe, since raw ERA5 cannot produce hydro inflow
    )
```

A thin dispatcher routes `calculate_capacity_factors` / `calculate_hydro_inflow` to the matching processor but writes the *same* canonical output path — so `rule all`, the GCS upload, and the model builder are untouched. The pure `entsoe`+`entsoe` combination delegates to the original code *byte-for-byte* (a hard backward-compatibility rule); any other combination composes wind/solar from `res_source` with RoR from `ror_source` into the canonical schema. Every processor is *fail-soft*: a missing input yields a valid empty file + a warning, never a crashed DAG.

5 The demand side: load curves & the load factor

In French, *facteur de charge* covers two distinct English objects. The **capacity factor** (§2) is a *generation*-side property: output over nameplate capacity. The **load factor** is the *demand*-side mirror: for a load curve $L(t)$ over a period,

$$\mathrm{LF} = \frac{\bar{L}}{L_{\max}} \qquad \text{(average load over peak load, in } (0, 1]\text{)}, \tag{5}$$

a measure of how *flat* the demand is — LF = 1 would be perfectly flat; a peaky, heating-driven curve (e.g. France in winter) has a lower LF. It is not fetched from any dataset: it is a *property of the load curve that DemandForge constructs*, and that construction is itself a per-dataset pipeline:

1. **ENTSO-E load** — the observed hourly national load $L(t)$ for a reference year.
2. **ERA5 × GHSL** — the *population-weighted* temperature $T_{\text{pop}}(t) = \sum_c \text{pop}_c\, T_c(t) / \sum_c \text{pop}_c$ (ERA5 2 m temperature on the grid cells where people actually live — the temperature the load responds to, not the area average).
3. **Thermosensitivity regression** — per hour-of-day, linear regressions of load against temperature below a winter threshold T_w and above a summer threshold T_s (thresholds chosen by maximising the combined R^2), giving heating and cooling slopes in MW/°C.
4. **Decomposition** — the load is split into additive components,

$$L(t) = B(t) \;+\; \beta_w \left(T_w - T_{\text{pop}}(t)\right)^+ \;+\; \beta_s \left(T_{\text{pop}}(t) - T_s\right)^+ \;+\; \text{EV}(t), \tag{6}$$

   i.e. baseload + winter-thermosensitive (heating) + summer-thermosensitive (cooling) + the ERAA EV charging profile (day-of-week aligned). Stored as the `combined_load_*.parquet` reference file.
5. **Projection** (`project_load_curve`) — each component is scaled *independently* to its target-year energy (growth rate or absolute target) and re-summed.

The load factor then *emerges* from the component mix — which is exactly why the decomposition matters: growing the thermosensitive share makes the curve peakier (LF ↓), heat-pump electrification reshapes winters, and EV charging adds its own profile. In the SupplyForge models the distinction is live: a node given a flat `demand_TWh` has LF = 1 (the simplification used in the EU greenfield demonstrator), whereas a node given a DemandForge `demand_profile` carries the realistic load factor (the FR–BE regional model). Adequacy-relevant results (peaking capacity, storage value, load shedding) are sensitive to this choice.

6 Packages & how they interact

SupplyForge (supply) and DemandForge (demand) are **sibling, decoupled** packages — neither imports the other. Each fetches its own raw data, harmonises it, and emits inputs for the same `pommes_craft` / POMMES optimisation, where supply technologies and demand series finally meet (Figure 1). They share several upstream sources (ERA5, ERAA, GHSL/GISCO geometry) and all the conventions (per-{country,year} files, Parquet, GCS caching, the `pommes_craft` library).

What each package fetches.

- **SupplyForge `fetch/`** — ENTSO-E (generation, installed capacity, unavailability, hydro storage, day-ahead prices, net-transfer capacities), ERAA (`eraa_study`), PECD (`fetch/pecd`), raw ERA5 (`fetch/era5`), OpenStreetMap (`fetch/osm`: plants, grid, industry), Ember (`ember_ntc`), and ENSPRESO biomass (`fetch/enspreso` → biomethane).
- **DemandForge `fetch/`** (the updated package at `paper_projects/demanfroge_update`) — ENTSO-E load curves, ERA5 2 m temperature, GHSL population (Europe-wide + country-scoped tiles), ERAA EV segments, industrial production (JRC-IDEES / TYNDP / USGS / Eurostat), country + NUTS geometry (GISCO), maritime bunkering weights, *and the new regional-evaluation sources*: facility-level industry (**Climate TRACE** as primary, E-PRTR / GEM / Hotmaps as complements), Eurostat regional statistics (population, GVA, EV stock via the dissemination API), national energy balances (`nrg_bal_c` gas/heat/electricity), national vehicle-fleet registries (FR SDES, DE KBA, ES DGT), and LNG/gas-storage locations (Wikidata).

How they meet. SupplyForge's `process` layer turns weather into the canonical capacity-factor / inflow / availability files (this note's subject); `grid_model` + `create_pommes_craft_model` assemble those into the supply side of a `pommes_craft` model. DemandForge's `load_projection`

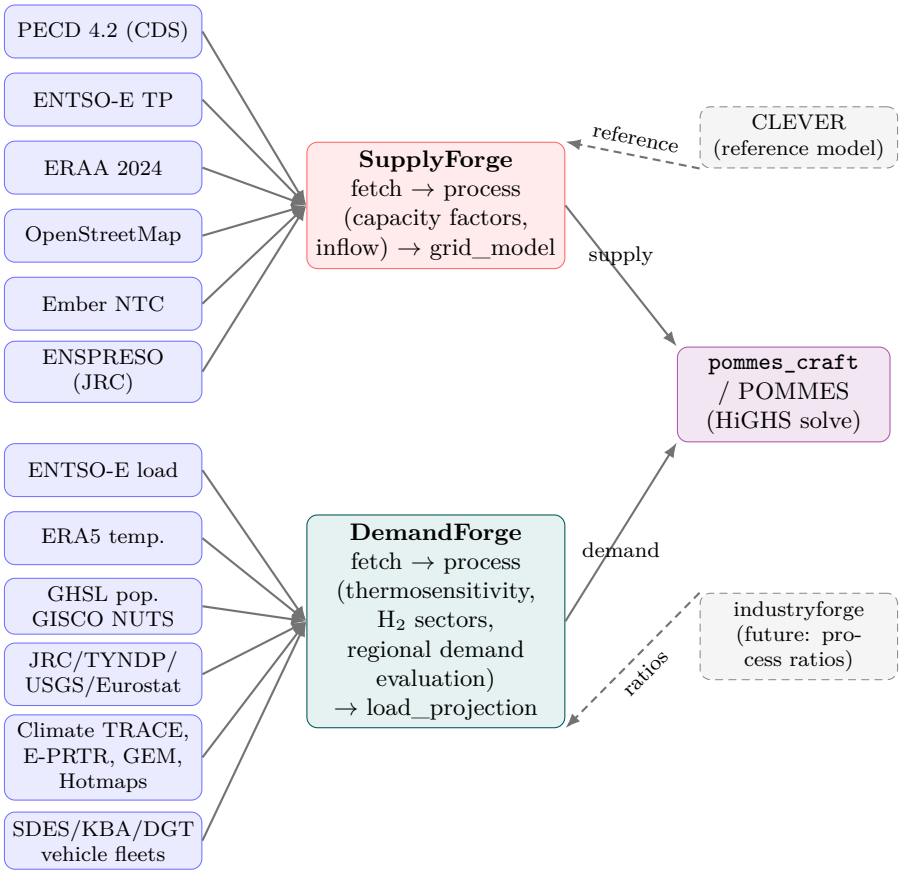

Figure 1: The two packages are decoupled and meet only at the POMMES model. SupplyForge builds the supply side (weather → capacity factors → techs); DemandForge builds the demand side (load + industry → projected demand). CLEVER is a read-only reference; the planned *industryforge* will own the industrial process ratios.

produces the electricity load curve and the industrial hydrogen demand (country level, unchanged), and its new **regional layer** — `process/demand_evaluation.evaluate_regional_demand` — now implements Phase 1 of the regional-demand specification: a bottom-up, base-year evaluation of *four* carriers (electricity, hydrogen, methane, heat) at NUTS-2, facility, and ∼1 km-grid granularity, with the national sum reconciled (reported, not clamped) against the country figures. The two packages are joined only inside POMMES (and via GCS-cached Parquets); the regional evaluation is the bridge that replaces the hand-picked demand weights in SupplyForge's regional models, and its forward *projection* (Phase 2) remains the user-interactive step.

7 Cross-cutting conventions

- **Leap years** — PECD/ERA5 drop Feb 29 to land on exactly 8760 h; the ENTSO-E inflow path produces 8784 h in leap years and the consumer truncates.
- **Clipping** — PECD/ERA5 clip CF to $[0, 1]$ and log exceedances; ENTSO-E is left unclipped (a CF > 1 flags a data anomaly).
- **Weather-year axis** — the `WS` column carries the climate/weather year (or the ERAA scenario), so several years can live in one file and be selected downstream.
- **Inflow is shape-only** — normalised by its max downstream, so only the profile matters.
- **Same keys everywhere** — per-{country, year} Parquet, GCS-cached, consumed unchanged by the model builders.

Credits

SupplyForge by **Yassine Abdelouadoud** (yassine.abdelouadoud@minesparis.psl.eu); the PECD/ERA5 source-switch, the capacity-factor processors and this note by **Simon Brigode** (simon.brigode@ehess.fr). Data: PECD 4.2 & ENSPRESO (JRC, Copernicus CDS), ENTSO-E Transparency, ERAA 2024, ERA5 (Copernicus), OpenStreetMap (ODbL), Ember, Eurostat GISCO, Climate TRACE (CC BY 4.0), E-PRTR/EEA, Hotmaps, Global Energy Monitor.